

# MSA algorytm genetyczny

Julianna Żurczak, Dawid Kubiak

01.06.2026

## 1 Opis problemu i algorytmu

### 1.1 Czym jest MSA

Dopasowanie wielu sekwencji (ang. *Multiple Sequence Alignment*, MSA) polega na ustawieniu zbioru sekwencji względem siebie tak, żeby litery powiązane ze sobą ewolucyjnie lub funkcjonalnie trafiły do tych samych kolumn. W naszym projekcie pracujemy na DNA, więc sekwencje to po prostu napisy nad alfabetem  $\{A, C, G, T\}$ .

Sekwencje prawie nigdy nie mają tej samej długości, dlatego w dopasowaniu wolno wstawiać znaki przerwy (luki), zapisywane jako „–”. Luka odpowiada insercji albo delecji powstałej w toku ewolucji. Po dołożeniu luk wszystkie sekwencje mają równą długość i można je ułożyć jako wiersze tablicy, w której każda kolumna zbiera litery (albo luki) ze wszystkich sekwencji.

Jakość dopasowania mówi, jak dobrze sekwencje są ustawione względem siebie: im więcej zgodnych liter w jednej kolumnie i im mniej drogich luk, tym lepiej. To pojęcie trzeba jednak zapisać liczbowo, jako funkcję celu, którą algorytm maksymalizuje. MSA jest trudne obliczeniowo (dla funkcji typu *sum-of-pairs* znalezienie optimum jest NP-trudne), więc zamiast metody dokładnej używamy metaheurystyki i zadowalamy się rozwiązaniem dobrym, choć niekoniecznie najlepszym.

### 1.2 Funkcja celu

Jakość mierzymy metodą sumy par (*sum-of-pairs*, SP). Dla  $k$  sekwencji bierzemy wszystkie  $\binom{k}{2}$  pary wierszy i dodajemy do siebie oceny dopasowań w parach:

$$SP = \sum_{1 \leq i < j \leq k} \text{score}(w_i, w_j),$$

gdzie  $\text{score}(w_i, w_j)$  to ocena pary wierszy  $i$  oraz  $j$ , liczona kolumna po kolumnie. Pojedyncze porównanie w kolumnie punktujemy tak:

| sytuacja                                   | punkty |
|--------------------------------------------|--------|
| zgodność liter ( <i>match</i> )            | +1     |
| niezgodność liter ( <i>mismatch</i> )      | −1     |
| otwarcie przerwy ( <i>gap open</i> )       | −2     |
| przedłużenie przerwy ( <i>gap extend</i> ) | −1     |
| przerwa naprzeciw przerwy                  | 0      |

Ważne jest tu rozróżnienie otwarcia i przedłużenia luki. Jeśli w danej parze wierszy luka zaczyna się dopiero w tej kolumnie (w poprzedniej, po tej samej stronie, jeszcze jej nie było), liczymy ją jako otwarcie ( $-2$ ). Jeśli luka ciągnie się już z poprzedniej kolumny, jest tańsza i liczymy ją jako przedłużenie ( $-1$ ). Dlatego kolumn nie da się oceniać całkiem osobno: przy karze za lukę trzeba zajrzeć do poprzedniej kolumny tej samej pary. To tak zwana kara afiniczna i lepiej pasuje do biologii, bo jedna dłuższa insercja jest bardziej prawdopodobna niż kilka krótkich.

Algorytm maksymalizuje SP. Wynik bywa ujemny i nie ma to znaczenia: liczy się tylko to, że z dwóch dopasowań lepsze jest to z wyższą wartością funkcji celu.

### 1.3 Reprezentacja rozwiązania

| Dopasowanie MSA       |               | Reprezentacja binarna                                                                                                                       |
|-----------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| w1: A G T C - G T A G | $\Rightarrow$ | $\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$ |
| w2: A - T C G T C - G |               |                                                                                                                                             |
| w3: - - - G T A G - - |               |                                                                                                                                             |

Rysunek 1: Przekształcenie tradycyjnego dopasowania sekwencji na macierz binarną stosowaną w algorytmie.

Pojedyncze rozwiązanie (dopasowanie) zapisujemy jako macierz binarną o  $k$  wierszach (po jednym na sekwencję) i wspólnej liczbie kolumn  $L$ . Wartość 1 na pozycji  $[i][j]$  oznacza, że w kolumnie  $j$  sekwencja  $i$  ma literę, a 0 oznacza lukę. Strukturę dopasowania (rozmieszczenie luk) oddzielamy więc od samych liter, które odczytujemy z oryginalnej sekwencji, biorąc je po kolei dla każdej jedynek w wierszu.

Rozwiązanie jest dopuszczalne, gdy spełnia dwa warunki:

1. liczba jedynek w wierszu  $i$  jest dokładnie równa długości sekwencji  $i$  (żadnej litery nie gubimy ani nie dodajemy),
2. żadna kolumna nie składa się z samych zer (kolumna samych luk niczego nie wnosi, a tylko obniża wynik, więc takie kolumny usuwamy).

Taki zapis jest wygodny, bo operatory genetyczne mogą działać wprost na macierzy zer i jedynek, a zamiana na czytelne dopasowanie z literami i myślnikami to już tylko prosta operacja pomocnicza.

### 1.4 Wybrana metaheurystyka – algorytm genetyczny

Jako metaheurystykę wybraliśmy algorytm genetyczny (GA). U nas pojedynczy osobnik to *całe dopasowanie*, a nie jedna sekwencja, więc chromosomem jest opisana wyżej macierz binarna. Funkcją przystosowania (*fitness*) jest wprost wartość SP. Algorytm trzyma populację dopasowań i z pokolenia na pokolenie poprawia ją selekcją, krzyżowaniem i mutacją, pilnując przy tym, żeby nie zgubić najlepszych rozwiązań.

Przy projektowaniu operatorów szybko zauważyliśmy, że zwykłe „bitowe” krzyżowanie i mutacja łatwo psują dopasowanie: potrafią zmienić liczbę liter w wierszu albo poprzestawiać ich kolejność. Dlatego dobraliśmy operatory tak, żeby same z siebie dawały rozwiązania dopuszczalne i nie wymagały losowego „naprawiania” znaków, które i tak rozbiłoby dobre fragmenty przekazane od rodziców.

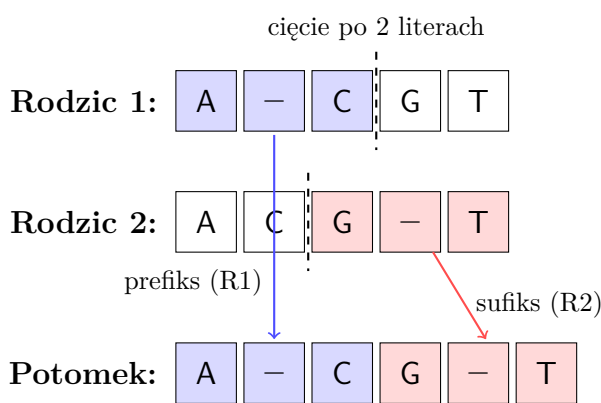
## Populacja początkowa

Populację startową losujemy. Dla każdej sekwencji robimy wiersz o szerokości najdłuższej sekwencji powiększonej o mały zapas na luki, wstawiamy tyle jedynek, ile sekwencja ma liter, resztę wypełniamy zerami i mieszamy. Kolumny z samych zer usuwamy. Takie dopasowania są słabe i zróżnicowane, dzięki czemu GA ma realne pole do poprawy.

## Selekcja

Rodziców wybieramy turniejem: losujemy  $k$  osobników i bierzemy z nich najlepszego. Metoda jest prosta, a rozmiarem turnieju  $k$  można regulować nacisk selekcyjny.

## Krzyżowanie



Rysunek 2: Krzyżowanie tnie rodziców po literach, a nie po kolumnach. Tutaj cięcie wypada po drugiej literze, więc w Rodzicu 1 przypada po kolumnie 3, a w Rodzicu 2 po kolumnie 2 (różne kolumny!). Potomek bierze prefiks z Rodzica 1 i sufiks z Rodzica 2; każda litera obu sekwencji trafia do potomka dokładnie raz, w niezmienionej kolejności.

Krzyżowanie to nasz najważniejszy operator i zaprojektowaliśmy je tak, żeby samo z siebie dawało poprawne dopasowanie, bez żadnej naprawy. Zamiast ciąć rodziców po kolumnach (co rozjechałoby liczbę liter w wierszach), tniemy ich po *literach*. Dla wybranego punktu cięcia, osobno w każdym wierszu, bierzemy z pierwszego rodzica początek z pewną liczbą liter, a brakujące litery dobieramy z drugiego rodzica. Skoro litery z prefiksu i sufiksu zawsze sumują się do długości sekwencji, potomek z definicji ma w każdym wierszu właściwą liczbę liter w niezmienionej kolejności. Nie musimy więc niczego odwracać na siłę i przekazujemy prawdziwy układ luk od obu rodziców. Na koniec wyrównujemy długości wierszy luką na końcu i usuwamy kolumny złożone z samych luk. Z dwóch rodziców robimy w ten sposób dwoje potomków, zamieniając między nimi rolę prefiksu i sufiksu.

## Mutacja

Mutacja przesuwa luki lokalnie. Z zadaniem prawdopodobieństwem losujemy wiersz i lukę w tym wierszu, po czym zamieniamy ją miejscami z sąsiednią pozycją (przesunięcie o jedną kolumnę w lewo lub w prawo). Treść sekwencji się przy tym nie zmienia, ruszamy tylko luki, więc dopasowanie dalej jest dopuszczalne, a algorytm może stopniowo dociągać ustawienie wspólnych fragmentów.

## Elityzm

Żeby nie tracić najlepszych rozwiązań, stosujemy elityzm: ustaloną liczbę najlepszych osobników przepisujemy bez zmian do następnego pokolenia. Dzięki temu jakość najlepszego dopasowania w populacji nigdy nie spada.

## 1.5 Warunek stopu

Algorytm zatrzymujemy po ustalonej z góry liczbie pokoleń. Warunek jest prosty i przewidywalny czasowo, co ułatwia porównywanie uruchomień przy tym samym budżecie obliczeń.

## 1.6 Najważniejsze parametry

| parametr             | wartość domyślna | znaczenie                                 |
|----------------------|------------------|-------------------------------------------|
| rozmiar populacji    | 30               | liczba dopasowań w populacji              |
| liczba pokoleń       | 120              | warunek stopu                             |
| prawdop. krzyżowania | 0,8              | szansa na krzyżowanie pary rodziców       |
| prawdop. mutacji     | 0,3              | szansa na mutację potomka                 |
| rozmiar turnieju     | 3                | nacisk selekcyjny                         |
| elityzm              | 2                | liczba osobników przepisywanych bez zmian |

Powyższe wartości potraktowaliśmy jako punkt wyjścia; ich wpływ na wynik sprawdzamy w części eksperymentalnej.

## 2 Część eksperymentalna

W tej części opisujemy przeprowadzone eksperymenty. Wykonaliśmy cztery rodzaje testów: (1) pomiar jakości dopasowania na instancjach dwóch rozmiarów (tabela wyników), (2) obserwację przebiegu zbieżności w kolejnych pokoleniach, (3) testy wpływu trzech parametrów algorytmu oraz (4) porównanie jakości na klasach o różnej trudności. Każdy wynik liczymy z wielu uruchomień (10 na konfigurację) i podajemy wartości średnie wraz z rozrzutem, ponieważ pojedynczy przebieg algorytmu losowego nie jest miarodajny.

### 2.1 Zbiory testowe

Instancje generujemy sztucznie, z jednej wspólnej sekwencji „przodka”. Z niego tworzymy kolejne sekwencje, wprowadzając losowe podstawienia oraz insercje i delecje. Dzięki temu sekwencje w jednej instancji są ze sobą spokrewnione i ich dopasowanie w ogóle ma sens – inaczej niż przy sekwencjach czysto losowych, gdzie wstawianie luk tylko obniża wynik i optimum sprowadza się do dosunięcia wszystkiego do lewej.

Przygotowaliśmy dwie klasy instancji różniące się rozmiarem:

- małe: 5 sekwencji o długości około 30,
- średnie: 6 sekwencji o długości około 60.

W każdej klasie jest po 5 instancji (razem 10). Generujemy je z ustalonych ziaren losowych, więc są powtarzalne między uruchomieniami.

Oprócz podziału według rozmiaru przygotowaliśmy też trzy *klasy trudności*, które różnią się podobieństwem sekwencji. Wszystkie mają ten sam rozmiar (6 sekwencji, długość około 50), a zmieniamy tylko nasilenie mutacji wprowadzanych względem przodka:

- łatwe – sekwencje bardzo podobne (małe tempo podstawień i indeli),
- średnie – umiarkowanie podobne,
- trudne – mało podobne (duża „odległość ewolucyjna”).

Po 5 instancji na klasę pozwala to pokazać, jak jakość działania algorytmu zależy od trudności danych. Do testów parametrów używamy natomiast osobnego, małego, ale zróżnicowanego zbioru 6 instancji (po dwie z każdej klasy trudności, w małym rozmiarze), żeby wniosków nie wyciągać z jednej instancji.

## 2.2 Sposób pomiaru wyników

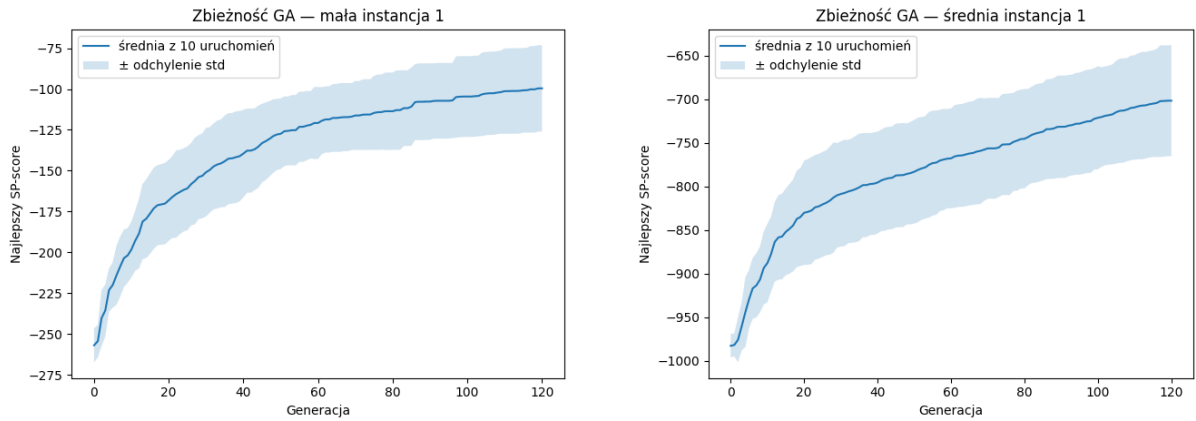
Główną miarą jakości jest wartość funkcji celu SP po wyczerpaniu budżetu obliczeniowego, czyli po 120 pokoleniach. Algorytm jest losowy, więc pojedynczy przebieg niewiele mówi – każdą instancję uruchamiamy 10 razy i podajemy:

- średni wynik końcowy,
- odchylenie standardowe wyników końcowych,
- najlepszy wynik z 10 przebiegów,
- średnią poprawę względem rozwiązania początkowego (najlepszego osobnika w populacji startowej),
- średni czas działania.

## 2.3 Wyniki

| klasa   | inst. | start (śr.) | koniec (śr.) | najlepszy | std  | poprawa | czas [s] |
|---------|-------|-------------|--------------|-----------|------|---------|----------|
| mała    | 1     | −254,9      | −115,6       | −16       | 50,3 | 139,3   | 0,4      |
| mała    | 2     | −223,3      | −101,5       | −57       | 31,4 | 121,8   | 0,4      |
| mała    | 3     | −246,1      | −114,3       | −69       | 29,6 | 131,8   | 0,4      |
| mała    | 4     | −263,1      | −109,2       | −66       | 27,6 | 153,9   | 0,4      |
| mała    | 5     | −215,2      | −79,0        | −2        | 48,6 | 136,2   | 0,4      |
| średnia | 1     | −984,2      | −753,0       | −685      | 38,3 | 231,2   | 1,1      |
| średnia | 2     | −809,8      | −565,2       | −477      | 44,5 | 244,6   | 1,0      |
| średnia | 3     | −829,5      | −580,8       | −499      | 48,2 | 248,7   | 3,0      |
| średnia | 4     | −854,4      | −536,7       | −408      | 58,1 | 317,7   | 3,3      |
| średnia | 5     | −855,7      | −614,7       | −517      | 62,0 | 241,0   | 3,0      |

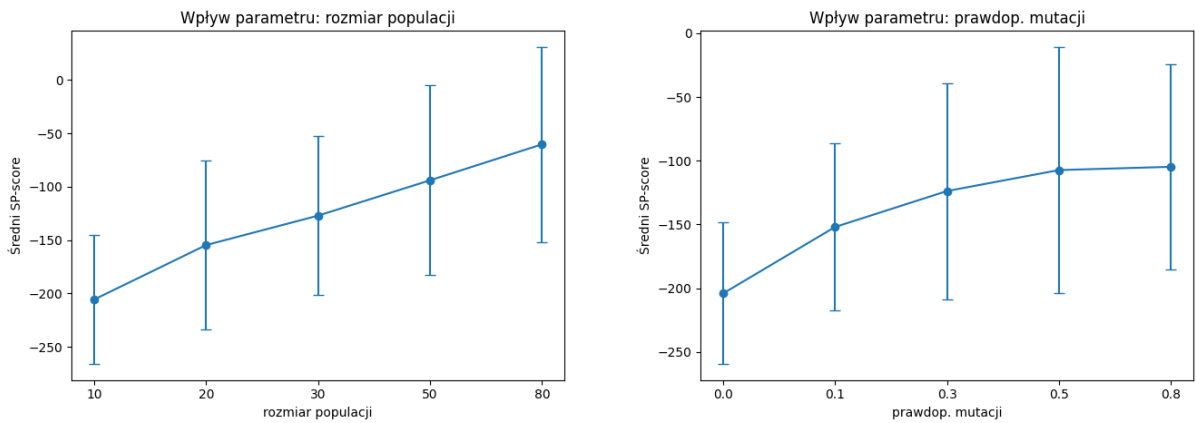
Tabela 1: Wyniki podstawowe: po 5 instancji w klasie małej i średniej, 10 uruchomień na instancję, parametry domyślne. Kolumny: średni wynik startowy i końcowy (średnie z 10 uruchomień), najlepszy wynik z 10 uruchomień, odchylenie standardowe wyników końcowych, średnia poprawa względem startu oraz średni czas pojedynczego uruchomienia.



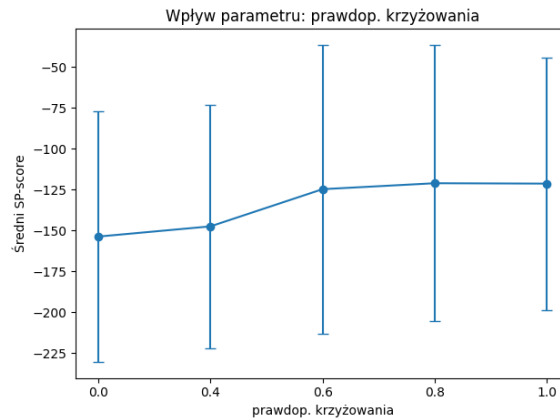
Rysunek 3: Uśredniony przebieg zbieżności z 10 uruchomień: mała instancja 1 (po lewej) i średnia instancja 1 (po prawej). Oś X: numer pokolenia; oś Y: najlepszy dotąd SP-score uśredniony po 10 uruchomieniach, a wstążka to odchylenie standardowe. Najlepszy wynik rośnie najszybciej na początku, a potem się stabilizuje.

## 2.4 Testy parametrów

Sprawdziliśmy wpływ trzech parametrów: rozmiaru populacji, prawdopodobieństwa mutacji oraz prawdopodobieństwa krzyżowania. W każdym teście zmieniamy jeden parametr (5 wartości), a pozostałe trzymamy na wartościach domyślnych. Każdą konfigurację uruchamiamy na całym zbiorze strojeniowym (6 zróżnicowanych instancji), po 10 razy na instancję, i raportujemy średni końcowy SP-score wraz z odchyleniem standardowym (słupki błędów na wykresach).



Rysunek 4: Wpływ rozmiaru populacji (po lewej) i prawdopodobieństwa mutacji (po prawej). Zbiór strojeniowy (6 instancji), 10 uruchomień na instancję. Oś X: testowana wartość parametru; oś Y: średni końcowy SP-score po 120 pokoleniach (średnia ze wszystkich uruchomień i instancji), słupki błędów to odchylenie standardowe.

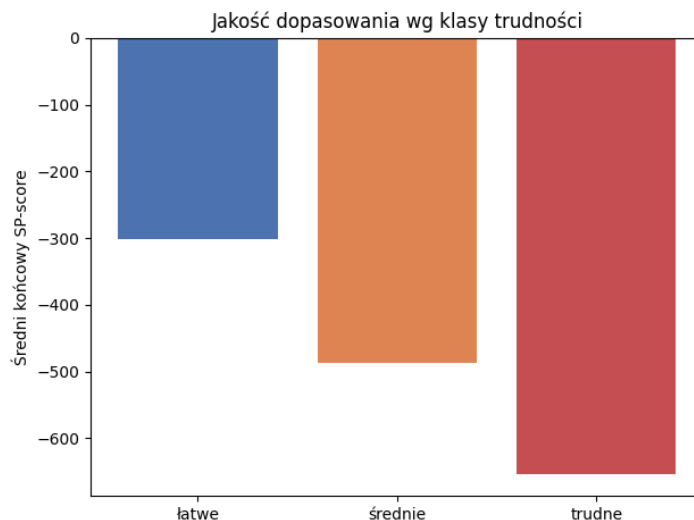


Rysunek 5: Wpływ prawdopodobieństwa krzyżowania. Warunki jak wyżej: zbiór strojenowy (6 instancji), 10 uruchomień na instancję; oś Y to średni końcowy SP-score, słupki błędów to odchylenie standardowe.

Z wykresów widać kilka rzeczy. Większy rozmiar populacji systematycznie poprawia wynik – więcej osobników to większa różnorodność i większa szansa, że selekcja znajdzie dobre fragmenty. Poprawa jest jednak okupiona dłuższym czasem obliczeń i rosnącym rozrzutem, więc nie zwiększaliśmy populacji w nieskończoność. Prawdopodobieństwo mutacji ma wyraźny wpływ: przy wartości 0,0 (brak mutacji) wynik jest najgorszy, co potwierdza, że to właśnie mutacja przesuwająca luki napędza lokalną poprawę; zwiększanie jej pomaga, choć rośnie też wariancja. Prawdopodobieństwo krzyżowania działa najsłabiej ze wszystkich trzech – wynik lekko rośnie wraz z nim, ale różnice są niewielkie w stosunku do rozrzutu, więc trudno tu mówić o silnym efekcie.

## 2.5 Wpływ klasy trudności

Na koniec sprawdziliśmy, jak algorytm radzi sobie z instancjami o różnym podobieństwie sekwencji. Dla każdej z trzech klas (łatwe, średnie, trudne) uruchomiliśmy GA na 5 instancjach, po 10 razy, przy parametrach domyślnych.



Rysunek 6: Średni końcowy SP-score dla trzech klas trudności. Po 5 instancji na klasę, 10 uruchomień na instancję, parametry domyślne. Oś Y to średnia ze wszystkich uruchomień i instancji danej klasy. Im mniej podobne sekwencje, tym niższy wynik.

Wynik jest zgodny z intuicją: im sekwencje są mniej podobne (większa „odległość ewolucyjna”), tym trudniej je dobrze dopasować i tym niższy końcowy SP-score. Dla sekwencji bardzo podobnych wspólne fragmenty są wyraźne i łatwo je ustawić w jednej kolumnie; dla sekwencji rozbieżnych jest mniej zgodnych liter, więcej koniecznych luk, a przestrzeń sensownych dopasowań robi się trudniejsza do przeszukania.

## 2.6 Komentarz i wnioski

Na każdej instancji algorytm poprawia rozwiązanie startowe – średnio o około 137 punktów dla instancji małych i o około 257 dla średnich. Poprawa jest więc widoczna i powtarzalna, a nie przypadkowa.

Odchylenie standardowe wyników (rzędu 28–62) pokazuje, że poszczególne przebiegi różnią się między sobą. To naturalne dla metaheurystyki, która startuje z losowej populacji i podejmuje losowe decyzje – właśnie dlatego uśredniamy wynik po 10 uruchomieniach, zamiast ufać jednemu. Widać też, że najlepszy pojedynczy przebieg potrafi wyraźnie przebić średnią (np. mała instancja 5 osiągnęła  $-2$  przy średniej  $-79,0$ ), więc w praktyce opłaca się uruchamiać algorytm wielokrotnie i brać najlepszy wynik.

Na instancjach średnich wynik bezwzględny jest niższy, ale bezwzględna poprawa większa. Dłuższe i liczniejsze sekwencje dają więcej luk do ustawienia, czyli większe pole do poprawy, ale jednocześnie trudniej dojść do dobrego dopasowania w tych samych 120 pokoleniach. Rośnie też czas obliczeń – z około 0,4 s na instancję małą do około 1–3 s na średnią, zależnie od długości sekwencji w danej instancji.

Wykresy zbieżności potwierdzają typowe dla GA zachowanie: największy zysk pojawia się w pierwszych kilkudziesięciu pokoleniach, po czym krzywa się spłaszcza. Algorytm

nie dochodzi do wyniku zerowego ani dodatniego na wszystkich instancjach i nie był to nasz cel – prosty operator mutacji (przesunięcie pojedynczej luki) ma ograniczoną siłę. Najważniejsze jest to, że w wyznaczonym czasie metaheurystyka konsekwentnie i mierzalnie poprawia rozwiązanie wyjściowe.